

Assignment 5 – SQL QUERIES

Name: Aryan Ligade

Prn-124b1f155

class/div-Sy(A4)

Case Study: Employee Performance Appraisal System

1. Aim

To write a PL/SQL program using a cursor, along with control structures (LOOP/IF) and exception handling, to process and display records from a database table.

2. Problem Statement (Case Study)

Write cursor code using control structures and exception handling

3. Database Design

1. User Entity

Stores details of users.

- user_id → INT, Primary Key
- name → VARCHAR(50), NOT NULL
- email → VARCHAR(100), UNIQUE
- phone → VARCHAR(15)
- city → VARCHAR(50)

Each user is uniquely identified by user_id.

◆ 2. Subscription Entity

Stores subscription details of users.

- subscription_id → INT, Primary Key
- user_id → INT, Foreign Key referencing User

- plan_name → VARCHAR(50)
- status → VARCHAR(20) (Active / Inactive)
- start_date → DATE
- end_date → DATE

One user can have multiple subscriptions.

◆ 3. Payment Entity

Stores payment details.

- payment_id → INT, Primary Key
- user_id → INT, Foreign Key referencing User
- subscription_id → INT, Foreign Key referencing Subscription
- amount → DECIMAL(10,2)
- payment_date → DATE
- payment_status → VARCHAR(20) (Paid / Pending / Failed)

Tracks all subscription payments.

◆ 4. Watch_History Entity

Stores user viewing history.

- watch_id → INT, Primary Key
- user_id → INT, Foreign Key referencing User
- content_name → VARCHAR(100)
- watch_date → DATE
- duration → INT (in minutes)

Used for cursor + WHILE loop program.

4. Database Schema

```
Command Prompt - mysql -u X + v

mysql> USE StreamingDB;
Database changed
mysql> CREATE TABLE User (
  -> user_id INT AUTO_INCREMENT PRIMARY KEY,
  -> name VARCHAR(100) NOT NULL,
  -> email VARCHAR(150) NOT NULL UNIQUE,
  -> phone VARCHAR(15)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Subscription_Plan (
  -> plan_id INT AUTO_INCREMENT PRIMARY KEY,
  -> plan_name VARCHAR(50) NOT NULL,
  -> price DECIMAL(8,2) NOT NULL
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Subscription (
  -> subscription_id INT AUTO_INCREMENT PRIMARY KEY,
  -> start_date DATE,
  -> end_date DATE,
  -> status VARCHAR(20) DEFAULT 'Active',
  -> user_id INT,
  -> plan_id INT,
  -> FOREIGN KEY (user_id) REFERENCES User(user_id),
  -> FOREIGN KEY (plan_id) REFERENCES Subscription_Plan(plan_id)
  -> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE TABLE Payment (
  -> payment_id INT AUTO_INCREMENT PRIMARY KEY,
  -> payment_date DATE,
  -> amount DECIMAL(8,2),
  -> payment_method VARCHAR(20),
  -> payment_status VARCHAR(20),
  -> subscription_id INT,
  -> FOREIGN KEY (subscription_id) REFERENCES Subscription(subscription_id) );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Content (
  -> content_id INT AUTO_INCREMENT PRIMARY KEY,
  -> title VARCHAR(200) NOT NULL,
  -> genre VARCHAR(50),
  -> duration INT
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Watch_History (
  -> history_id INT AUTO_INCREMENT PRIMARY KEY,
  -> watch_date DATE,
  -> user_id INT,
  -> content_id INT,
  -> FOREIGN KEY (user_id) REFERENCES User(user_id),
  -> FOREIGN KEY (content_id) REFERENCES Content(content_id)
  -> );
Query OK, 0 rows affected (0.03 sec)
```

5. SQL Inputs and Outputs

-- Sample Data(inserting)

```
mysql> -- Sample Data
mysql> INSERT INTO Subscription_Plan VALUES (1,'Basic',149),(2,'Standard',499),(3,'Premium',649);
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO User (name, email, phone) VALUES
-> ('Rahul Patil', 'rahul@gmail.com', '9876543210'),
-> ('Sneha Kulkarni', 'sneha@gmail.com', '9823456780'),
-> ('Amit Sharma', 'amit@yahoo.com', '9512345678');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Content (title, genre, duration) VALUES
-> ('Pushpa 2', 'Action', 180), ('3 Idiots', 'Comedy', 170), ('RRR', 'Action', 182);
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Subscription (start_date, end_date, status, user_id, plan_id)
-> VALUES ('2025-01-01', '2025-01-31', 'Active', 1, 3),
-> ('2025-01-10', '2025-02-09', 'Active', 2, 2),
-> ('2024-12-01', '2024-12-31', 'Expired', 3, 1);
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Payment (payment_date, amount, payment_method, payment_status, subscription_id)
-> VALUES
-> ('2025-01-01', 649, 'UPI', 'Success', 1),
-> ('2025-01-10', 499, 'Card', 'Success', 2),
-> ('2024-12-01', 149, 'Cash', 'Pending', 3);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Watch_History (watch_date, user_id, content_id) VALUES
-> ('2025-01-05', 1, 1), ('2025-01-06', 1, 3),
-> ('2025-01-11', 2, 2), ('2025-01-12', 2, 1);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

3. Program 1 – Display All Users

Aim: Use a cursor to display all users. Concepts: Cursor, LOOP, IF (control structure), SQLEXCEPTION handler (exception handling).

```
mysql> CREATE PROCEDURE sp_display_users()
-> BEGIN
->     -- Variable Declaration
->     DECLARE v_id INT;
->     DECLARE v_name VARCHAR(100);
->     DECLARE v_email VARCHAR(150);
->     DECLARE v_done INT DEFAULT FALSE;
->
->     -- Cursor Declaration
->     DECLARE cur_users CURSOR FOR
->     SELECT user_id, name, email FROM `User`;
->
->     -- NOT FOUND Handler
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
->
->     -- Exception Handling
->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->         SELECT 'Error occurred while fetching users.' AS Message;
->     END;
->
->     OPEN cur_users;
->
->     user_loop: LOOP
->         FETCH cur_users INTO v_id, v_name, v_email;
->
->         IF v_done THEN
->             LEAVE user_loop;
->         END IF;
->
->         SELECT v_id AS UserID, v_name AS Name, v_email AS Email;
->     END LOOP;
->
->     CLOSE cur_users;
-> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
```

Output:

```
mysql>
mysql> DELIMITER ;
mysql> CALL sp_display_users();
+-----+-----+-----+
| UserID | Name      | Email      |
+-----+-----+-----+
|      1 | Rahul Patil | rahul@gmail.com |
+-----+-----+-----+
1 row in set (0.01 sec)

+-----+-----+-----+
| UserID | Name      | Email      |
+-----+-----+-----+
|      2 | Sneha Kulkarni | sneha@gmail.com |
+-----+-----+-----+
1 row in set (0.01 sec)

+-----+-----+-----+
| UserID | Name      | Email      |
+-----+-----+-----+
|      3 | Amit Sharma | amit@yahoo.com |
+-----+-----+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.02 sec)
```

Program 2 – Display Subscriptions with Status Check

Aim: Use a cursor to display subscriptions. Use IF-ELSE to show a message based on status. Concepts: Cursor, IF-ELSE (control

structure), SQLEXCEPTION han

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE sp_display_subscriptions()
-> BEGIN
->     -- Variable Declaration
->     DECLARE v_sub_id INT;
->     DECLARE v_name VARCHAR(100);
->     DECLARE v_plan VARCHAR(50);
->     DECLARE v_status VARCHAR(20);
->     DECLARE v_remark VARCHAR(30);
->     DECLARE v_done INT DEFAULT FALSE;
->
->     -- Cursor Declaration
->     DECLARE cur_subs CURSOR FOR
->     SELECT s.subscription_id, u.name, sp.plan_name, s.status
->     FROM Subscription s
->     JOIN `User` u ON s.user_id = u.user_id
->     JOIN Subscription_Plan sp ON s.plan_id = sp.plan_id;
->
->     -- NOT FOUND Handler
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
->
->     -- Exception Handler
->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->         SELECT 'Error occurred while fetching subscriptions.' AS Message;
->     END;
->
->     OPEN cur_subs;
->
->     sub_loop: LOOP
->         FETCH cur_subs INTO v_sub_id, v_name, v_plan, v_status;
->
->         IF v_done THEN
->             LEAVE sub_loop;
->         END IF;
->
->         -- IF-ELSE logic
```

er (exception h

```
->     END;
->
->     OPEN cur_subs;
->
->     sub_loop: LOOP
->         FETCH cur_subs INTO v_sub_id, v_name, v_plan, v_status;
->
->         IF v_done THEN
->             LEAVE sub_loop;
->         END IF;
->
->         -- IF-ELSE logic
->         IF v_status = 'Active' THEN
->             SET v_remark = 'Currently Active';
->         ELSE
->             SET v_remark = 'Not Active';
->         END IF;
->
->         SELECT v_sub_id AS SubID,
->                v_name AS UserName,
->                v_plan AS Plan,
->                v_status AS Status,
->                v_remark AS Remark;
->     END LOOP;
->
->     CLOSE cur_subs;
-> END$$
Query OK, 0 rows affected (0.01 sec)
```

Output:

```
mysql>
mysql> DELIMITER ;
mysql> CALL sp_display_subscriptions();
+-----+-----+-----+-----+-----+
| SubID | UserName      | Plan    | Status | Remark          |
+-----+-----+-----+-----+-----+
|      1 | Rahul Patil   | Premium | Active | Currently Active |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

+-----+-----+-----+-----+-----+
| SubID | UserName      | Plan    | Status | Remark          |
+-----+-----+-----+-----+-----+
|      2 | Sneha Kulkarni | Standard | Active | Currently Active |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

+-----+-----+-----+-----+-----+
| SubID | UserName      | Plan    | Status | Remark          |
+-----+-----+-----+-----+-----+
|      3 | Amit Sharma   | Basic   | Expired | Not Active      |
+-----+-----+-----+-----+-----+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.02 sec)
```

Program 3 – Payment Status Check using CASE Aim: Use a cursor to read all payments. Use CASE to display a message for each payment status. Concepts: Cursor, CASE (control structure), SQLEXCEPTION handler (exception handling).

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE sp_check_payments()
-> BEGIN
->   -- Variable Declaration
->   DECLARE v_pay_id INT;
->   DECLARE v_name VARCHAR(100);
->   DECLARE v_amount DECIMAL(8,2);
->   DECLARE v_method VARCHAR(20);
->   DECLARE v_pstatus VARCHAR(20);
->   DECLARE v_msg VARCHAR(50);
->   DECLARE v_done INT DEFAULT FALSE;
->
->   -- Cursor Declaration
->   DECLARE cur_pay CURSOR FOR
->   SELECT p.payment_id, u.name, p.amount,
->         p.payment_method, p.payment_status
->   FROM Payment p
->   JOIN Subscription s ON p.subscription_id = s.subscription_id
->   JOIN 'User' u ON s.user_id = u.user_id;
->
->   -- NOT FOUND Handler
->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
->
->   -- Exception Handler
->   DECLARE EXIT HANDLER FOR SQLEXCEPTION
->   BEGIN
->     SELECT 'Error occurred while processing payments.' AS Message;
->   END;
-> END;
```



```

-> OPEN cur_pay;
->
-> pay_loop: LOOP
->     FETCH cur_pay INTO v_pay_id, v_name, v_amount,
->                         v_method, v_pstatus;
->
->     IF v_done THEN
->         LEAVE pay_loop;
->     END IF;
->
->     -- CASE control structure
->     CASE v_pstatus
->         WHEN 'Success' THEN SET v_msg = 'Payment Received';
->         WHEN 'Pending' THEN SET v_msg = 'Awaiting Confirmation';
->         WHEN 'Failed' THEN SET v_msg = 'Payment Failed - Retry';
->         ELSE SET v_msg = 'Unknown Status';
->     END CASE;
->
->     SELECT v_pay_id AS PaymentID,
->            v_name AS UserName,
->            v_amount AS Amount,
->            v_method AS Method,
->            v_pstatus AS Status,
->            v_msg AS Message;
->
-> END LOOP;
->
-> CLOSE cur_pay;
-> END$$
Query OK, 0 rows affected (0.01 sec)

```

Output:

```

mysql>
mysql> DELIMITER ;
mysql> CALL sp_check_payments();
+-----+-----+-----+-----+-----+-----+
| PaymentID | UserName | Amount | Method | Status | Message |
+-----+-----+-----+-----+-----+-----+
| 1 | Rahul Patil | 649.00 | UPI | Success | Payment Received |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

+-----+-----+-----+-----+-----+-----+
| PaymentID | UserName | Amount | Method | Status | Message |
+-----+-----+-----+-----+-----+-----+
| 2 | Sneha Kulkarni | 499.00 | Card | Success | Payment Received |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

+-----+-----+-----+-----+-----+-----+
| PaymentID | UserName | Amount | Method | Status | Message |
+-----+-----+-----+-----+-----+-----+
| 3 | Amit Sharma | 149.00 | Cash | Pending | Awaiting Confirmation |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.03 sec)

```

6. Program 4 – Watch History with WHILE Loop Aim: Use a cursor with a WHILE loop to read watch history and count total watches. Concepts: Cursor, WHILE loop (control structure), SQLEXCEPTION handler (exception handling).

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE sp_watch_history_report()
-> BEGIN
->     -- Variable Declaration
->     DECLARE v_uname VARCHAR(100);
->     DECLARE v_title VARCHAR(200);
->     DECLARE v_date DATE;
->     DECLARE v_count INT DEFAULT 0;
->     DECLARE v_done INT DEFAULT FALSE;
->
->     -- Cursor Declaration
->     DECLARE cur_wh CURSOR FOR
->     SELECT u.name, c.title, wh.watch_date
->     FROM Watch_History wh
->     JOIN `User` u ON wh.user_id = u.user_id
->     JOIN Content c ON wh.content_id = c.content_id
->     ORDER BY wh.watch_date DESC;
->
->     -- NOT FOUND Handler
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
->
->     -- Exception Handler
->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->         SELECT 'Error occurred while reading watch history.' AS Message;
->     END;
->
->     OPEN cur_wh;
```

```
->
-> OPEN cur_wh;
->
-> watch_loop: LOOP
->     FETCH cur_wh INTO v_username, v_title, v_date;
->
->     IF v_done THEN
->         LEAVE watch_loop;
->     END IF;
->
->     SET v_count = v_count + 1;
->
->     SELECT v_count AS SrNo,
->           v_username AS UserName,
->           v_title AS ContentTitle,
->           v_date AS WatchDate;
-> END LOOP;
->
-> CLOSE cur_wh;
->
-> -- Total count
-> SELECT v_count AS TotalWatchRecords;
-> END$$
Query OK, 0 rows affected (0.01 sec)
```

Output:

```
mysql> DELIMITER ;
mysql> CALL sp_watch_history_report();
```

SrNo	UserName	ContentTitle	WatchDate
1	Sneha Kulkarni	Pushpa 2	2025-01-12

```
1 row in set (0.01 sec)
```

SrNo	UserName	ContentTitle	WatchDate
2	Sneha Kulkarni	3 Idiots	2025-01-11

```
1 row in set (0.01 sec)
```

SrNo	UserName	ContentTitle	WatchDate
3	Rahul Patil	RRR	2025-01-06

```
1 row in set (0.02 sec)
```

SrNo	UserName	ContentTitle	WatchDate
4	Rahul Patil	Pushpa 2	2025-01-05

```
1 row in set (0.02 sec)
```

TotalWatchRecords
4

```
1 row in set (0.03 sec)
```

```
Query OK, 0 rows affected (0.03 sec)
```

Program 5 – Validate and Insert Subscription (SIGNAL)

Aim: Use a cursor to check if a user already has an active subscription before inserting a new one. Raise an error using SIGNAL if duplicate found. Concepts: Cursor, IF (control structure), SIGNAL SQLSTATE + DECLARE HANDLER (exception handling).

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE sp_add_subscription(
    ->     IN p_user_id INT,
    ->     IN p_plan_id INT,
    ->     OUT p_result VARCHAR(100)
    -> )
    -> BEGIN
    ->     -- Variable Declaration
    ->     DECLARE v_existing INT DEFAULT 0;
    ->     DECLARE v_sub_id INT;
    ->     DECLARE v_status VARCHAR(20);
    ->     DECLARE v_done INT DEFAULT FALSE;
    ->
    ->     -- Cursor Declaration
    ->     DECLARE cur_check CURSOR FOR
    ->     SELECT subscription_id, status
    ->     FROM Subscription
    ->     WHERE user_id = p_user_id;
    ->
    ->     -- NOT FOUND Handler
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
    ->
    ->     -- Exception Handler
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         SET p_result = 'ERROR: Could not create subscription.';
    ->     END;
    ->
```

```

-> OPEN cur_check;
->
-> check_loop: LOOP
->     FETCH cur_check INTO v_sub_id, v_status;
->
->     IF v_done THEN
->         LEAVE check_loop;
->     END IF;
->
->     IF v_status = 'Active' THEN
->         SET v_existing = 1;
->         LEAVE check_loop;
->     END IF;
-> END LOOP;
->
-> CLOSE cur_check;
->
-> -- Check condition
-> IF v_existing = 1 THEN
->     SET p_result = 'User already has an active subscription.';
-> ELSE
->     START TRANSACTION;
->
->     INSERT INTO Subscription
->     (start_date, end_date, status, user_id, plan_id)
->     VALUES
->     (CURRENT_DATE,
->      DATE_ADD(CURRENT_DATE, INTERVAL 30 DAY),
->      'Active',
->      p_user_id,
->      p_plan_id);
->
->     COMMIT;
->
->     SET p_result = CONCAT('SUCCESS: Subscription ID = ', LAST_INSERT_ID());
-> END IF;
->
-> END$$

```

Output:

```

mysql>
mysql> DELIMITER ;
mysql> CALL sp_add_subscription(3, 2, @res);
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT @res;
+-----+
| @res                                     |
+-----+
| SUCCESS: Subscription ID = 4 |
+-----+
1 row in set (0.00 sec)

mysql> CALL sp_add_subscription(1, 2, @res);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @res;
+-----+
| @res                                     |
+-----+
| User already has an active subscription. |
+-----+
1 row in set (0.00 sec)

```

7. Conclusion

The study and implementation of SQL queries using **Joins, Subqueries, Views, GROUP BY, and HAVING clauses** play a crucial role in efficient database management and data retrieval. Through this practical, we understood how relational databases organize data into multiple tables and how these tables can be logically connected using keys.

By using different types of **Joins** such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and others, we were able to combine data from multiple tables and extract meaningful information. Joins helped in understanding relationships between entities like Passenger, Flight, and Booking, making it easier to retrieve combined data in a single query.

The use of **Subqueries** enhanced the flexibility of SQL by allowing queries within queries. This helped in solving complex problems such as finding maximum values, filtering records based on aggregated results, and performing conditional checks dynamically. Correlated subqueries further improved query precision by executing row-wise operations.

The implementation of **Views** provided an abstraction layer over complex queries. Views simplified data access, improved security by restricting direct table access, and allowed reuse of frequently used queries without rewriting them.

Additionally, the use of **GROUP BY and HAVING clauses** enabled data aggregation and filtering of grouped results. These clauses were useful for generating summaries, such as counting passengers per flight or identifying flights with a high number of bookings.

Overall, this practical demonstrated how SQL DML operations combined with advanced querying techniques help in managing and analyzing large datasets efficiently. It also highlighted the importance of proper database design, normalization, and the use of keys to maintain data integrity and consistency.